

StudyAI — Full Project Documentation

(mirroring the CaptionAI sample's structure: Description → Scenarios → Architecture → Prerequisites → Milestones → Conclusion)

Project Description

StudyAI is an AI-powered study companion that helps college students convert their own uploaded course material into exam-ready learning resources. The platform includes an **Upload module** supporting PDF/DOCX/TXT files, a **Summary Generator** that extracts and condenses content using an LLM, and in-progress **Flashcard** and **Quiz Generator** modules for active recall and self-testing.

Utilizing Groq's API with a Llama model, StudyAI processes extracted text from PyMuPDF (fitz) to deliver fast, structured study content. Built with a React.js frontend and Flask backend, the platform gives students a single dashboard to manage uploads, review summaries, and track their revision progress — reducing the manual effort of converting raw lecture material into something testable.

Scenarios

Scenario 1: A student uploads a 40-page lecture PDF the night before an exam. The system extracts the text via PyMuPDF, sends it to the Groq API, and returns a concise structured summary in seconds — cutting down what would have been an hour of manual reading and note-taking.

Scenario 2: A student preparing for multiple subjects uploads separate files for each and uses the Dashboard to see all their materials in one place, switching between Summary pages for each subject without re-uploading or re-organizing notes manually.

Scenario 3 (in progress): Once Flashcards and Quiz generation are complete, a student will be able to take their uploaded material, generate a quiz automatically, attempt it, and get an immediate sense of their weak areas before the exam — closing the loop from "raw notes" to "self-assessed and ready."

Technical Architecture

Description: StudyAI uses a three-tier architecture. The frontend (React.js, React Router) provides page-based navigation across Home, Upload, Summary, Flashcards, Quiz, Planner, and Dashboard. The Flask backend handles file validation, text extraction, and orchestrates calls to the Groq API. Local file storage currently handles uploads, with UUID-based naming preventing collisions.

[React Frontend] → [Flask API (routes)] → [pdf_service.py: text extraction]

→ [groq_service.py: Groq API call]

→ [summary_service.py: response formatting]

↓

[uploads/ folder — local file storage]

(Note: no database is currently used — StudyAI stores files locally by UUID rather than in a persistent DB. Per the sample template's own instruction, an ER diagram is only required if a database is in use; StudyAI doesn't have one yet, so this can be honestly stated rather than fabricated.)

Pre-requisites

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Groq API Familiarity:** [Groq API Documentation](#)
- **React.js & React Router:** [React Documentation](#)
- **PyMuPDF (fitz):** [PyMuPDF Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** Node.js + npm (frontend), Python 3.8+ + pip (backend)

Project Workflow

Milestone 1: Architecture & Setup

- Set up Groq API key securely via .env
- Selected Llama model on Groq for summarization
- Defined frontend/backend structure (React pages + Flask routes/services)
- Set up dev environment: Flask, Flask-CORS, PyMuPDF, Groq SDK

Milestone 2: Core Feature Development

- Built Upload module: file validation, UUID naming, local storage
- Built Summary module: PDF text extraction → Groq API → structured summary response

Milestone 3: Backend Route Development

- Implemented /upload (POST) — accepts file, validates type, saves with UUID filename
- Implemented /summary (POST) — reads a saved file, extracts text, calls Groq, returns summary JSON

Milestone 4: Frontend Development

- Built Home, Upload, Summary, Dashboard pages with Burgundy theme (#800020)

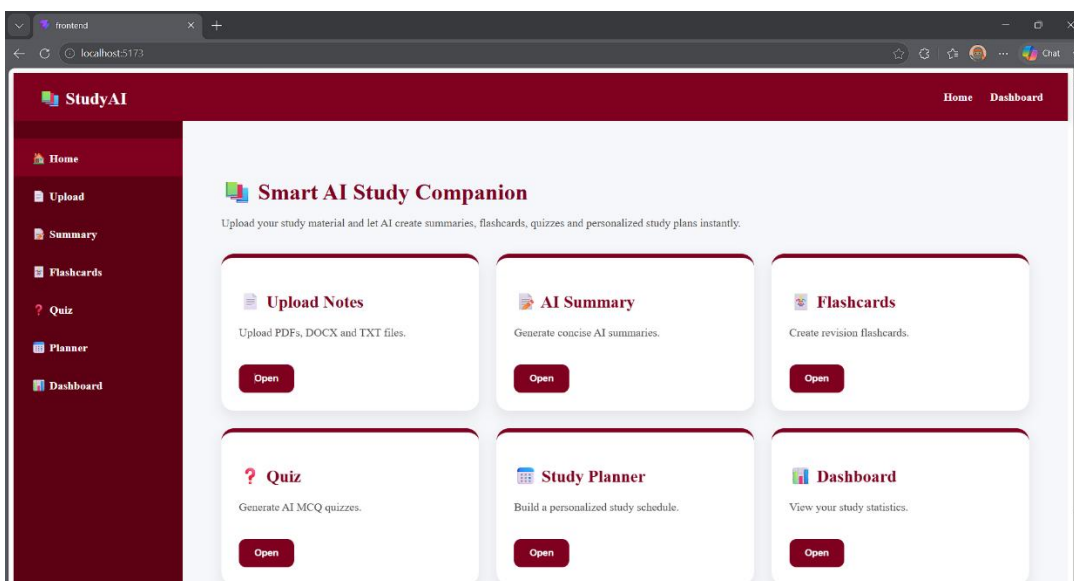
- Built reusable Navbar, Sidebar, and DashboardCard components
- Connected Summary page live to backend via api.js

Milestone 5: In Progress

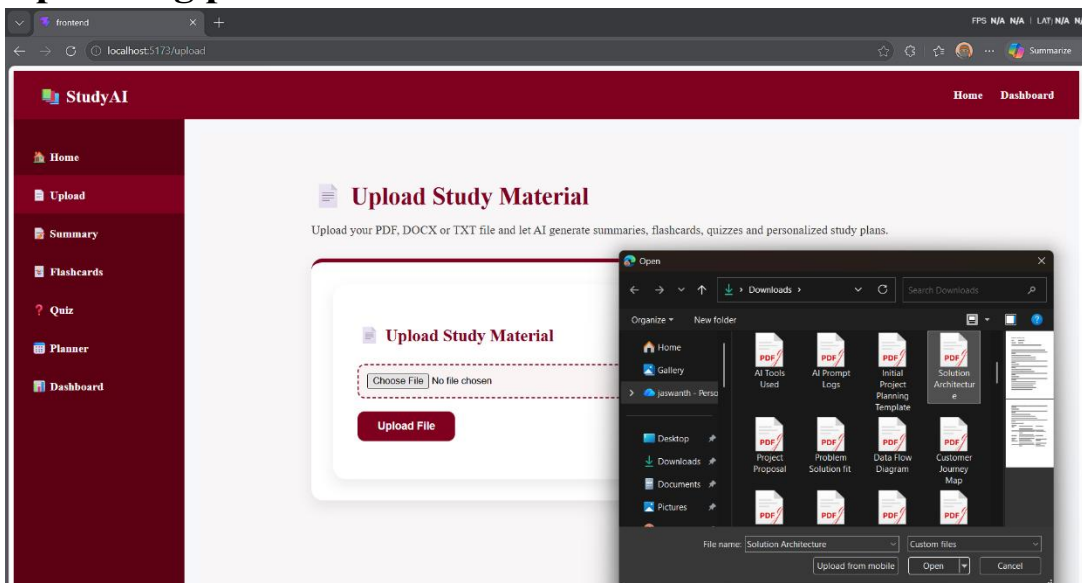
- Flashcards module (frontend + generation logic)
- Quiz module (frontend + generation logic)
- Study Planner

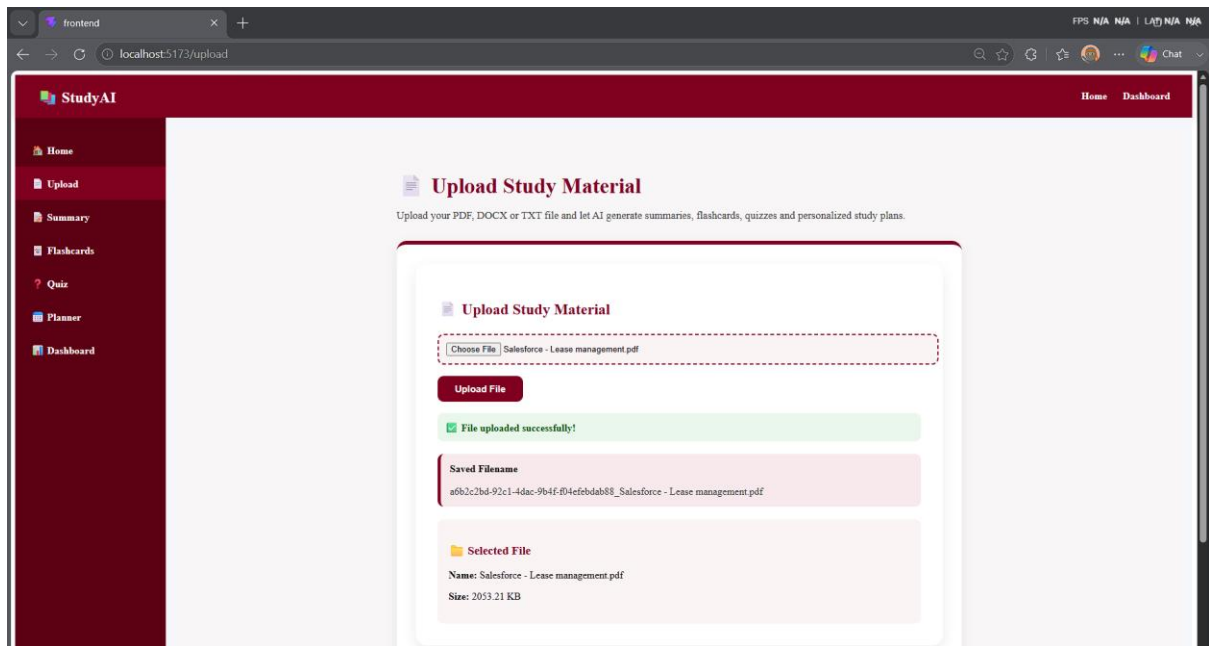
Screenshots:

Home Page :

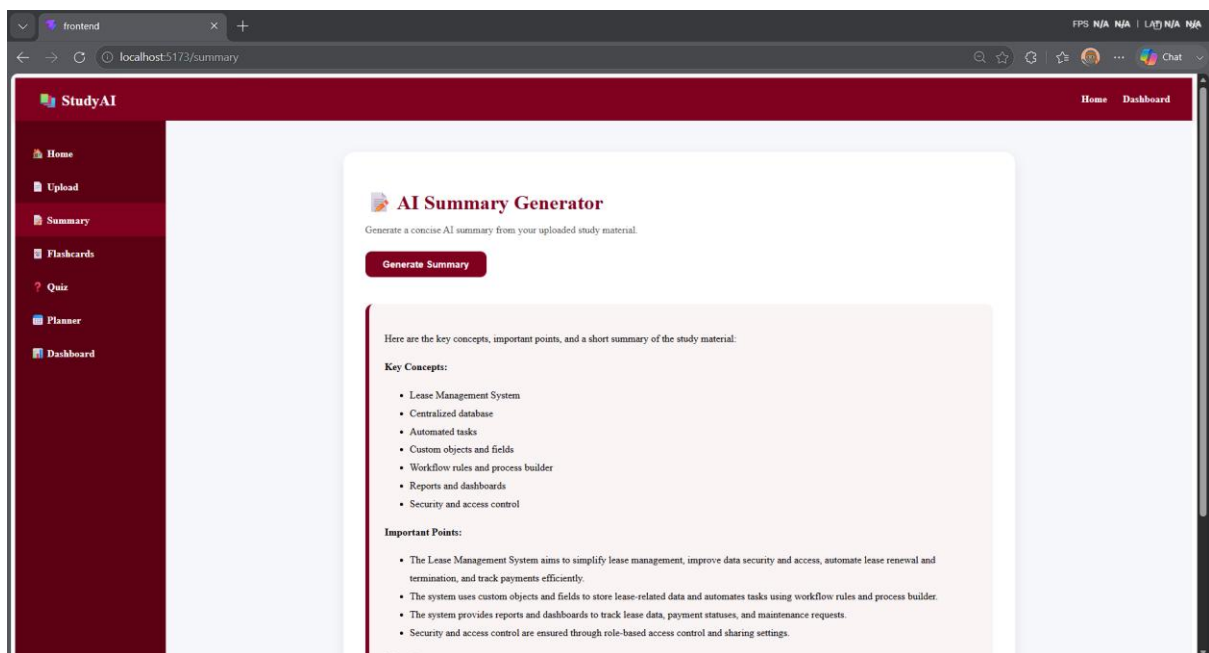


Uploading pdf:

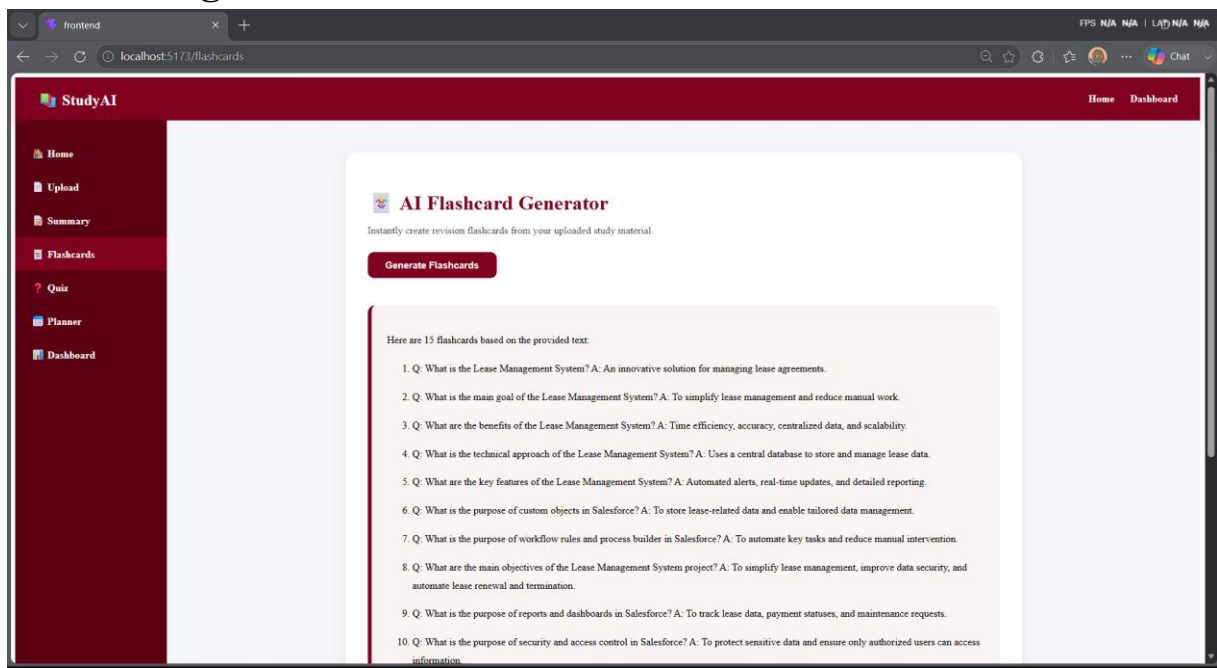




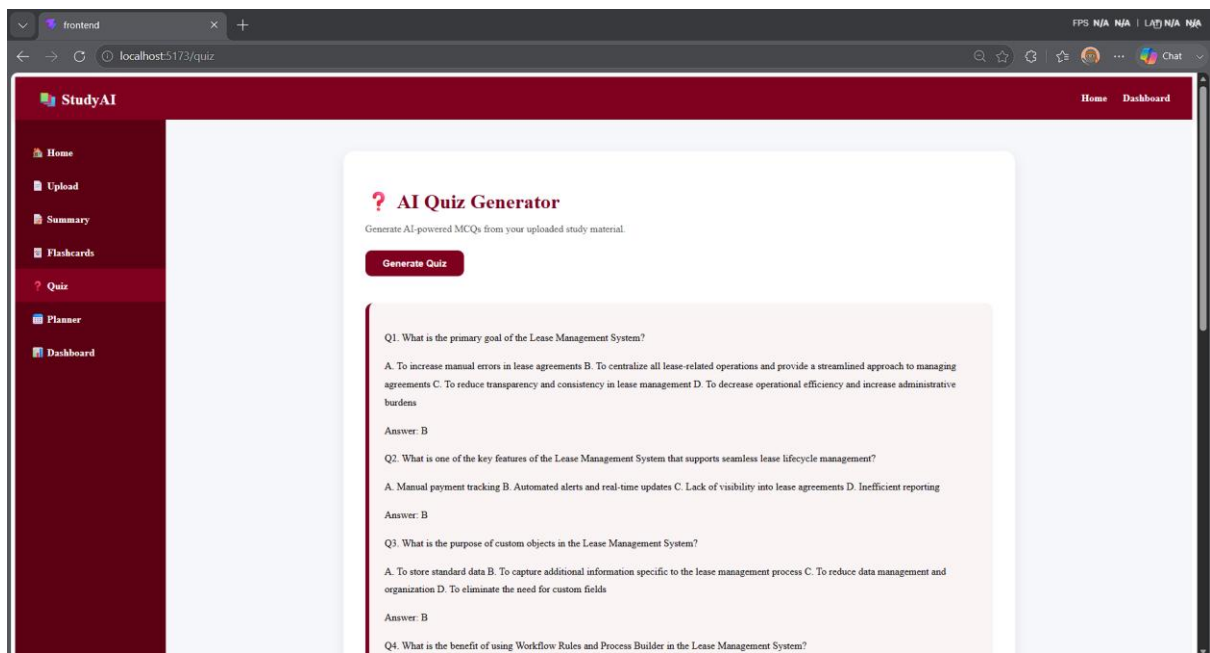
Summary generation:



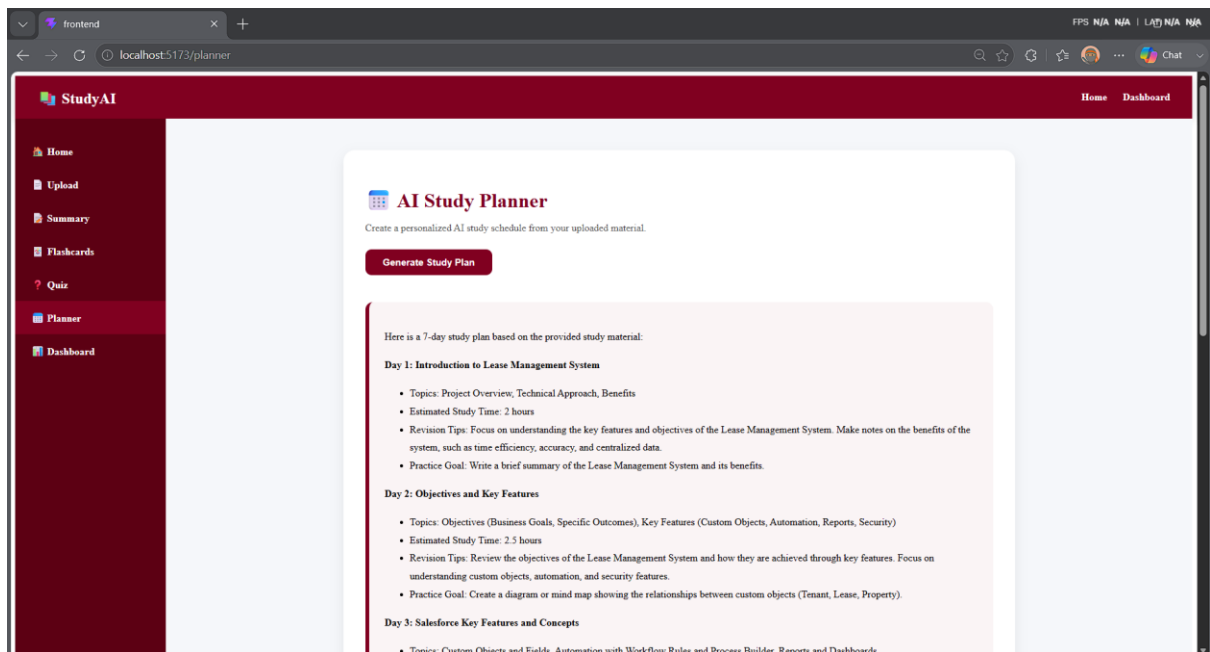
Flashcard generation



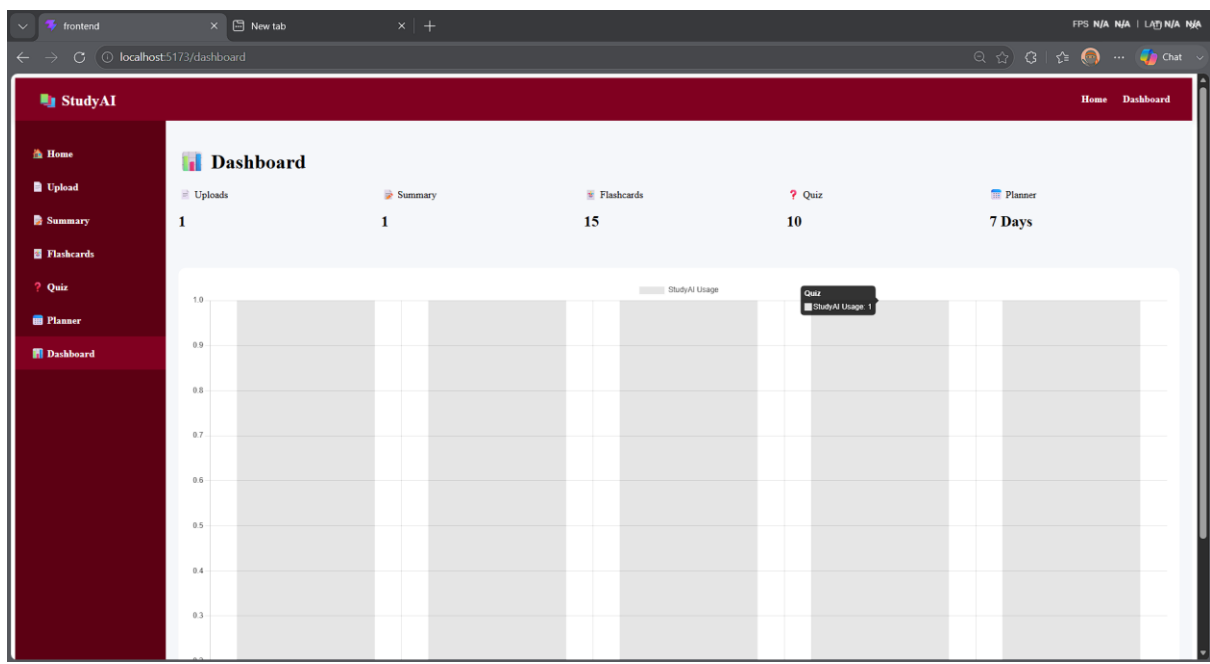
Quiz generation:



AI Study Planner



Dashboard



Milestone 6: Conclusion

Conclusion

StudyAI is a Flask + React platform that automates the conversion of a student's own uploaded material into exam-ready study content, using Groq's Llama model for fast summarization. With Upload and Summary fully functional and connected end-to-end, the project demonstrates a working AI content pipeline; Flashcards, Quiz, and Planner modules are in active development to complete the full study-prep loop. Future scope includes persistent storage via a database, user authentication, and progress tracking — turning StudyAI from a single-session tool into a full account-based study platform.